Lab 1 · Modelado OLTP → OLAP — Informe PayTrack

Autor: Eduardo Andrés Piñeros Manjarres · Entrega: viernes 23 de octubre de 2026

1. Diagnóstico del OLTP (Parte A)

Datos de prueba: **50 000** transacciones. Cada consulta se ejecutó con `EXPLAIN ANALYZE` sobre `oltp.*`.

Consulta	Tiempo total (ms)	Scan sobre transactions	Tipo de join	Agregación / sort	Filas procesadas
Q1 · monto por categoría (30 días)	22.937	Bitmap Heap Scan (vía Bitmap Index Scan en idx_txn_created_at)	Hash Join con merchants	HashAggregate + Sort	16 678
Q2 · tasa de fraude por segmento	13.556	Seq Scan	Hash Join con customers	HashAggregate + Sort	50 000
Q3 · volumen por día y hora (7 días)	5.004	Bitmap Heap Scan (vía idx_txn_created_at)	no aplica (sin join)	Sort (3 778 filas) + GroupAggregate	3 778

Q1:

"Sort (cost=1448.24..1448.44 rows=80 width=290) (actual time=22.803..22.806 rows=7 loops=1)"

" Sort Key: (sum(t.amount_cop)) DESC"

" Sort Method: quicksort Memory: 25kB"

" -> HashAggregate (cost=1444.51..1445.71 rows=80 width=290) (actual time=22.754..22.761 rows=7 loops=1)"

" Group Key: m.category"

" Batches: 1 Memory Usage: 24kB"

" -> Hash Join (cost=369.68..1319.26 rows=16700 width=226) (actual time=1.611..20.236 rows=16678 loops=1)"

" Hash Cond: ((t.merchant_id)::text = (m.merchant_id)::text)"

" -> Bitmap Heap Scan on transactions t (cost=357.88..1261.30 rows=16700 width=13) (actual time=1.553..18.160 rows=16678 loops=1)"

" Recheck Cond: (created_at >= (CURRENT_DATE - 30))"

" Filter: ((status)::text = 'approved'::text)"

" Rows Removed by Filter: 19"

" Heap Blocks: exact=569"

" -> Bitmap Index Scan on idx_txn_created_at (cost=0.00..353.70 rows=16721 width=0) (actual time=1.504..1.504 rows=16697 loops=1)"

" Index Cond: (created_at >= (CURRENT_DATE - 30))"

" -> Hash (cost=10.80..10.80 rows=80 width=256) (actual time=0.048..0.049 rows=8 loops=1)"

" Buckets: 1024 Batches: 1 Memory Usage: 9kB"

" -> Seq Scan on merchants m (cost=0.00..10.80 rows=80 width=256) (actual time=0.035..0.036 rows=8 loops=1)"

"Planning Time: 1.856 ms"

"Execution Time: 22.937 ms"

Q2:

"Sort (cost=1595.70..1595.71 rows=3 width=54) (actual time=13.499..13.501 rows=3 loops=1)"

" Sort Key: (round(((100.0 * (sum(CASE WHEN t.is_fraud THEN 1 ELSE 0 END))::numeric) / (count(*))::numeric), 2)) DESC"

" Sort Method: quicksort Memory: 25kB"

" -> HashAggregate (cost=1595.61..1595.68 rows=3 width=54) (actual time=13.471..13.475 rows=3 loops=1)"

" Group Key: c.segment"

" Batches: 1 Memory Usage: 24kB"

" -> Hash Join (cost=19.25..1220.61 rows=50000 width=7) (actual time=0.213..8.916 rows=50000 loops=1)"

" Hash Cond: ((t.customer_id)::text = (c.customer_id)::text)"

" -> Seq Scan on transactions t (cost=0.00..1069.00 rows=50000 width=9) (actual time=0.004..2.378 rows=50000 loops=1)"

" -> Hash (cost=13.00..13.00 rows=500 width=14) (actual time=0.200..0.201 rows=500 loops=1)"

" Buckets: 1024 Batches: 1 Memory Usage: 31kB"

" -> Seq Scan on customers c (cost=0.00..13.00 rows=500 width=14) (actual time=0.004..0.136 rows=500 loops=1)"

"Planning Time: 0.881 ms"

"Execution Time: 13.556 ms"

Q3:

"GroupAggregate (cost=965.67..1077.79 rows=3737 width=104) (actual time=4.315..4.854 rows=166 loops=1)"

" Group Key: (EXTRACT(dow FROM created_at)), (EXTRACT(hour FROM created_at))"

" -> Sort (cost=965.67..975.01 rows=3738 width=72) (actual time=4.278..4.374 rows=3778 loops=1)"

" Sort Key: (EXTRACT(dow FROM created_at)), (EXTRACT(hour FROM created_at))"

" Sort Method: quicksort Memory: 273kB"

" -> Bitmap Heap Scan on transactions t (cost=81.30..743.85 rows=3738 width=72) (actual time=0.343..1.968 rows=3778 loops=1)"

" Recheck Cond: (created_at >= (CURRENT_DATE - 7))"

" Filter: ((status)::text = 'approved'::text)"

" Rows Removed by Filter: 5"

" Heap Blocks: exact=569"

" -> Bitmap Index Scan on idx_txn_created_at (cost=0.00..80.37 rows=3743 width=0) (actual time=0.235..0.235 rows=3783 loops=1)"

" Index Cond: (created_at >= (CURRENT_DATE - 7))"

"Planning Time: 0.171 ms"

"Execution Time: 5.004 ms"

¿Qué domina cada plan y por qué lo obliga la 3FN?

- **Q1.** (22.937 ms) El atributo `category` vive en `merchants`, no en `transactions`, así que las 16 678 filas aprobadas de los últimos 30 días deben cruzarse con `merchants` mediante un Hash Join, y luego agruparse (HashAggregate) y ordenarse por `total_cop`. El filtro por `created_at` sí usa `idx_txn_created_at` (Bitmap Index Scan sobre ~33 % de las 50 000 filas), pero el índice no cubre `status` ni `amount_cop`: el Bitmap Heap Scan lee 569 bloques del heap y aplica `status = 'approved'` fila por fila (solo descarta 19). Es el plan más costoso de los tres porque combina scan, join, agregación y sort.

- **Q2.** (13.556 ms) No tiene filtro: hace un Seq Scan de las 50 000 filas de `transactions` y las cruza con `customers` (500 filas) en un Hash Join para obtener `segment`. Ningún índice ayuda porque la consulta necesita todas las filas; el costo crece linealmente con el histórico completo. En 3FN el segmento está separado del hecho, así que el join es inevitable.
- **Q3.** (5.004 ms) `EXTRACT(DOW ...)` y `EXTRACT(HOUR ...)` se calculan por cada una de las 3 778 filas filtradas y, como el `GROUP BY` es sobre expresiones, el plan debe ordenar esas 3 778 filas (Sort de 273 kB) antes del GroupAggregate en lugar de agregar directamente. Ningún índice puede resolver esa agrupación porque el modelo normalizado no almacena las partes de la fecha precalculadas.
- **Causa de fondo.** El esquema OLTP está optimizado para escribir un pago y consultarlo por clave primaria con mínima redundancia. Las preguntas analíticas hacen lo contrario: leen muchas filas, agregan y cruzan dimensiones.

Proyección a 500 000 transacciones diarias durante un año. Son 500 000 × 365 = 182,5 millones de filas, es decir **3 650 veces** más que con 50 000. Q2 lee toda la tabla, así que con crecimiento lineal pasaría de **13.556 ms** a aproximadamente **49 479 ms (≈ 49.5 s)**. Q1 y Q3 solo leen una ventana de 30 y 7 días, que a ese ritmo serían ~15 millones y ~3.5 millones de filas (≈ 900× lo actual): del orden de **~21 s** para Q1 y **~4.6 s** para Q3 (extrapolación lineal, solo como orden de magnitud). Además, a esa escala la tabla ya no cabe en memoria y los sorts y agregaciones empiezan a derramar a disco, por lo que los tiempos reales serían peores.

2. Decisiones de diseño del esquema estrella (Parte B)

Proceso de negocio y grano. Proceso: pagos de PayTrack y su detección de fraude. Grano de `fact_transactions`: **una fila por transacción** (`transaction_id`).

Dimensiones.

Dimensión	Atributos	Tipo	Observación
`dim_date`	año, trimestre, mes, semana, día, día de la semana, fin de semana, festivo CO	dada	Evita `EXTRACT` en tiempo de consulta
`dim_time`	hora, minuto, periodo del día	dada	Granularidad de minuto
`dim_customer`	`customer_id` (natural), ciudad, segmento, `income_band`, `valid_from`, `valid_to`, `is_current`	SCD Tipo 2	Sin nombre, email ni ingreso exacto
`dim_merchant`	`merchant_id` (natural), nombre, categoría, ciudad	Tipo 1	La categoría es el eje de Q1
`dim_payment_method`	`method_name` (natural),	Tipo 1	Agrupa en tarjeta /

Dimensión	Atributos	Tipo	Observación
	`method_type`		billetera / otro

Datos del OLTP que no llegan al DW. `name` y `email` del cliente (datos personales, sin valor analítico; el análisis se hace por segmento y ciudad). `monthly_income_cop` se reemplaza por `income_band` para no exponer el ingreso exacto y facilitar el análisis por rangos. `created_at` de la transacción se descompone en `date_key` y `time_key`. De `fraud_events`, `pattern` y `detected_at` se dejan fuera porque ese es otro grano (varios eventos por transacción) y las preguntas del negocio no los usan; solo se conserva el riesgo máximo como medida.

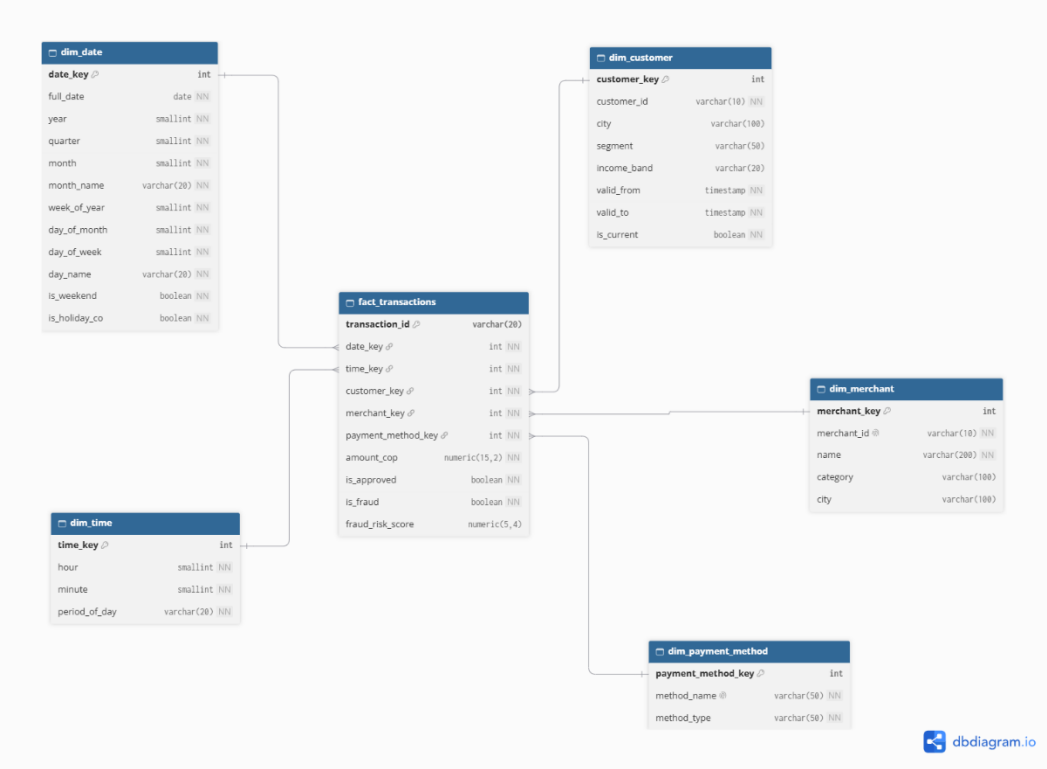
Medidas.

Medida	Aditividad	Cómo se agrega
`amount_cop`	Aditiva	`SUM` en cualquier dimensión
`is_approved`, `is_fraud`	Aditivas como conteo (flags)	`COUNT(*) FILTER (WHERE ...)` o `SUM(CASE ...)`
tasa de fraude	No aditiva (razón)	Se calcula como razón de sumas, nunca se promedian tasas
`fraud_risk_score`	No aditiva	`AVG` o `MAX`, no `SUM`

No hay medidas semiaditivas porque el modelo no guarda saldos ni inventarios.

Surrogate keys. (1) Con SCD2 un mismo `customer_id` tiene varias filas, así que la llave natural no puede ser la PK. (2) Aíslan al DW de cambios o reutilización de llaves en el sistema fuente. (3) Son enteros pequeños: joins e índices más compactos y rápidos que con `VARCHAR`. (4) Permiten manejar miembros desconocidos sin romper la integridad referencial.

Por qué SCD Tipo 2 en cliente. `segment`, `city` e `income_band` pueden cambiar (un cliente pasa de *mass* a *premium*). Si se sobrescribe, todo el historial del cliente se reatribuiría al segmento nuevo y Q2 (tasa de fraude por segmento) quedaría sesgada: el fraude ocurrido cuando era *mass* se contaría como *premium*. Con SCD2, cada hecho apunta a la versión vigente en la fecha de la transacción, y por eso Q2 **no** filtra por `is_current`.



3. Comparación de rendimiento OLTP vs OLAP (Parte D)

```
(paytrack-bigdata-labs) PS C:\Users\USUARIO\Documents\FILES\Alm. Grande Datos\big-data-labs-2026-2> uv run python labs/01-modelado/load_star.py
>>
✓ Esquema estrella creado
✓ dim_date: 91 filas
✓ dim_time: 1440 filas
✓ dim_customer: 500 filas
✓ dim_merchant: 8 filas
✓ dim_payment_method: 5 filas
✓ fact_transactions: 50000 filas

Validando la carga...
  ✓ dim_customer tiene un registro vigente por cliente
  ✓ dim_merchant tiene todos los comercios
  ✓ dim_payment_method tiene todos los métodos
  ✓ fact_transactions tiene una fila por transacción
  ✓ el monto total cuadra
  ✓ el número de fraudes cuadra
  ✓ el número de transacciones aprobadas cuadra
  ✓ toda transacción tiene una fecha y hora válidas en las dimensiones
  ✓ toda transacción tiene cliente, comercio y método en las dimensiones

¡Esquema estrella cargado y validado!
```

Resultados verificados con `check_queries.py`:

```
(paytrack-bigdata-labs) PS C:\Users\USUARIO\Documents\FILES\Alm. Grande Datos\big-data-labs-2026-2> uv run python labs/01-modelado/check_queries.py
>>
  ✓ Q1: 7 filas idénticas
  ✓ Q2: 3 filas idénticas
  ✓ Q3: 166 filas idénticas

3/3 consultas correctas
```

Consulta	OLTP (ms)	OLAP sin índices (ms)	OLAP con índices (ms)	Scan OLTP → OLAP	Mejora (OLTP ÷ OLAP)

Consulta	OLTP (ms)	OLAP sin índices (ms)	OLAP con índices (ms)	Scan OLTP → OLAP	Mejora (OLTP ÷ OLAP)
Q1	22.937	[MEDIR sin idx_fact_date]	18.667	Bitmap Heap Scan → Bitmap Heap Scan (idx_fact_date)	1.23×
Q2	13.556	Igual (el plan no usa índices)	12.145	Seq Scan → Seq Scan	1.12×
Q3	5.004	[MEDIR sin idx_fact_date]	2.851	Bitmap Heap Scan → Bitmap Heap Scan (idx_fact_date)	1.76×

Q1: "Sort (cost=1357.81..1358.01 rows=80 width=290) (actual time=18.474..18.477 rows=7 loops=1)"

" Sort Key: (sum(f.amount_cop)) DESC"

" Sort Method: quicksort Memory: 25kB"

" -> HashAggregate (cost=1354.08..1355.28 rows=80 width=290) (actual time=18.417..18.423 rows=7 loops=1)"

" Group Key: m.category"

" Batches: 1 Memory Usage: 24kB"

" -> Hash Join (cost=215.84..1230.60 rows=16464 width=226) (actual time=0.913..16.103 rows=16678 loops=1)"

" Hash Cond: (f.merchant_key = m.merchant_key)"

" -> Bitmap Heap Scan on fact_transactions f (cost=204.04..1173.29 rows=16464 width=12) (actual time=0.826..14.465 rows=16678 loops=1)"

" Recheck Cond: (date_key >= (to_char(((CURRENT_DATE - 30))::timestamp with time zone, 'YYYYMMDD'::text))::integer)"

" Filter: is_approved"

" Rows Removed by Filter: 19"

" Heap Blocks: exact=516"

" -> Bitmap Index Scan on idx_fact_date (cost=0.00..199.92 rows=16482 width=0) (actual time=0.763..0.764 rows=16697 loops=1)"

" Index Cond: (date_key >= (to_char(((CURRENT_DATE - 30))::timestamp with time zone, 'YYYYMMDD'::text))::integer)"

" -> Hash (cost=10.80..10.80 rows=80 width=222) (actual time=0.032..0.033 rows=8 loops=1)"

" Buckets: 1024 Batches: 1 Memory Usage: 9kB"

" -> Seq Scan on dim_merchant m (cost=0.00..10.80 rows=80 width=222) (actual time=0.020..0.021 rows=8 loops=1)"

"Planning Time: 2.094 ms"

"Execution Time: 18.667 ms"

Q2:

"Sort (cost=1540.70..1540.71 rows=3 width=54) (actual time=12.087..12.089 rows=3 loops=1)"

" Sort Key: (round(((100.0 * (sum(CASE WHEN f.is_fraud THEN 1 ELSE 0 END))::numeric) / (count(*))::numeric), 2)) DESC"

" Sort Method: quicksort Memory: 25kB"

" -> HashAggregate (cost=1540.61..1540.68 rows=3 width=54) (actual time=12.073..12.077 rows=3 loops=1)"

" Group Key: c.segment"

" Batches: 1 Memory Usage: 24kB"

" -> Hash Join (cost=17.25..1165.61 rows=50000 width=7) (actual time=0.141..7.387 rows=50000 loops=1)"

" Hash Cond: (f.customer_key = c.customer_key)"

" -> Seq Scan on fact_transactions f (cost=0.00..1016.00 rows=50000 width=5) (actual time=0.004..1.837 rows=50000 loops=1)"

" -> Hash (cost=11.00..11.00 rows=500 width=10) (actual time=0.108..0.109 rows=500 loops=1)"

" Buckets: 1024 Batches: 1 Memory Usage: 30kB"

" -> Seq Scan on dim_customer c (cost=0.00..11.00 rows=500 width=10) (actual time=0.005..0.062 rows=500 loops=1)"

"Planning Time: 0.968 ms"

"Execution Time: 12.145 ms"

Q3:

"Sort (cost=782.66..783.08 rows=168 width=44) (actual time=2.665..2.672 rows=166 loops=1)"

" Sort Key: d.day_of_week, t.hour"

" Sort Method: quicksort Memory: 34kB"

" -> HashAggregate (cost=774.35..776.45 rows=168 width=44) (actual time=2.581..2.603 rows=166 loops=1)"

" Group Key: d.day_of_week, t.hour"

" Batches: 1 Memory Usage: 160kB"

" -> Hash Join (cost=94.40..736.14 rows=3821 width=12) (actual time=0.548..2.053 rows=3778 loops=1)"

" Hash Cond: (f.time_key = t.time_key)"

" -> Hash Join (cost=53.00..684.68 rows=3821 width=14) (actual time=0.238..1.399 rows=3778 loops=1)"

" Hash Cond: (f.date_key = d.date_key)"

" -> Bitmap Heap Scan on fact_transactions f (cost=49.95..671.14 rows=3821 width=16) (actual time=0.197..1.045 rows=3778 loops=1)"

" Recheck Cond: (date_key >= (to_char(((CURRENT_DATE - 7))::timestamp with time zone, 'YYYYMMDD'::text))::integer)"

" Filter: is_approved"

" Rows Removed by Filter: 5"

" Heap Blocks: exact=516"

" -> Bitmap Index Scan on idx_fact_date (cost=0.00..48.99 rows=3825 width=0) (actual time=0.162..0.162 rows=3783 loops=1)"

" Index Cond: (date_key >= (to_char(((CURRENT_DATE - 7))::timestamp with time zone, 'YYYYMMDD'::text))::integer)"

" -> Hash (cost=1.91..1.91 rows=91 width=6) (actual time=0.021..0.021 rows=91 loops=1)"

" Buckets: 1024 Batches: 1 Memory Usage: 12kB"

" -> Seq Scan on dim_date d (cost=0.00..1.91 rows=91 width=6) (actual time=0.003..0.007 rows=91 loops=1)"

" -> Hash (cost=23.40..23.40 rows=1440 width=6) (actual time=0.295..0.297 rows=1440 loops=1)"

" Buckets: 2048 Batches: 1 Memory Usage: 73kB"

" -> Seq Scan on dim_time t (cost=0.00..23.40 rows=1440 width=6) (actual time=0.004..0.177 rows=1440 loops=1)"

"Planning Time: 1.003 ms"

"Execution Time: 2.851 ms"

Qué cambia en los planes.

- **Q1.** (22.937 → 18.667 ms, 1.23×) El plan conserva la misma forma (Bitmap Index Scan → Bitmap Heap Scan → Hash Join → HashAggregate → Sort), pero el filtro pasa de comparar un timestamp y un texto (`status = 'approved'`) a comparar un entero (`date_key`) y un booleano (`is_approved`), y el join es contra `dim_merchant` por clave entera. La tabla de hechos también es más angosta: el Bitmap Heap Scan lee 516 bloques frente a 569 del OLTP (~9 % menos).
- **Q2.** (13.556 → 12.145 ms, 1.12×) Sigue siendo un Seq Scan de toda la tabla, así que la mejora es modesta, como se esperaba. La ganancia viene de una fila de hechos más angosta (516 páginas en vez de 569; costo del Seq Scan 1016 vs 1069) y de un join con claves enteras contra `dim_customer`.
- **Q3.** (5.004 → 2.851 ms, 1.76×) Es la mayor mejora. Desaparece el cálculo de `EXTRACT` por fila y, sobre todo, el Sort de 3 778 filas (273 kB) previo a la agregación: el día de la semana y la hora salen de `dim_date` y `dim_time`, el HashAggregate reduce las 3 778 filas a 166 grupos y solo se ordenan esas 166 (34 kB). El costo es planificar dos joins más: el Planning Time sube de 0.171 a 1.003 ms, pero no anula la ganancia.
- **Lectura honesta.** Con 50 000 filas la mejora es de 1.12× a 1.76× (OLTP ÷ OLAP con índices): pequeña, porque todo cabe en memoria y las dimensiones son diminutas. El beneficio real del modelo dimensional se ve al escalar: menos cómputo por fila, filas más angostas y agregaciones sobre columnas ya preparadas.

Experimento con índices (TODO 5). Para cada índice se midió el plan con y sin él (`EXPLAIN ANALYZE`).

Índice	Consulta que debería usarlo	Plan sin índice	Plan con índice	¿Ayuda?
`idx_fact_date (date_key)`	Q1, Q3	Q1:26.305ms Q3:14.070ms	Bitmap Index Scan en Q1 (16 697 filas, 0.76 ms) y en Q3 (3 783 filas, 0.16 ms)	Se usa en Q1 y Q3; confirmar el beneficio con la medición sin índice
`idx_fact_merchant (merchant_key)`	Q1	Mismo plan (no se usa)	No aparece: Hash Join contra dim_merchant (8 filas)	No
`idx_fact_customer (customer_key)`	Q2	Mismo plan (no se usa)	No aparece: Seq Scan de la tabla de hechos + Hash Join con dim_customer (500 filas)	No
`idx_fact_time (time_key)`	Q3	Mismo plan (no se usa)	No aparece: Hash Join con dim_time	No

Índice	Consulta que debería usarlo	Plan sin índice	Plan con índice	¿Ayuda?
			(1 440 filas)	
`idx_fact_fraud (parcial, WHERE is_fraud)`	ninguna	Mismo plan (no se usa)	No aparece: ninguna consulta filtra por is_fraud	No

¿Todos ayudan? ¿Por qué el optimizador prefiere a veces un Seq Scan?

- Un índice solo conviene cuando el filtro es **selectivo**. Q2 lee el 100 % de la tabla y el planificador usa Seq Scan. En Q1 el filtro devuelve ~33 % de las filas (16 697 de 50 000) y el planificador eligió un Bitmap Index Scan en vez de Seq Scan: un punto intermedio que ordena los accesos al heap antes de leerlos. Si el rango creciera hacia la totalidad de la tabla, el acceso aleatorio saldría más caro y volvería el Seq Scan.
- Los índices sobre claves foráneas (`merchant_key`, `customer_key`, `time_key`) no aparecen en ningún plan: los joins son Hash Join contra dimensiones pequeñas (8, 500, 1 440 y 91 filas) que se cargan en una tabla hash mientras se recorre la tabla de hechos, así que no hay búsquedas por clave que un índice pueda acelerar.
- `idx_fact_date` es el único índice que se usa: en Q3 selecciona el 7.6 % de la tabla (3 783 de 50 000 filas) y en Q1 el 33 %. Los datos abarcan 91 días (`dim_date` tiene 91 filas), por eso 30 días ya es una fracción importante. **[MEDIR: compara Q1 y Q3 con y sin este índice para cuantificar su aporte.]**
- `idx_fact_fraud` no se usa: ninguna de las tres consultas filtra por `is_fraud` (Q2 lo agrega con `CASE`, por lo que recorre toda la tabla). Un índice sin consultas que lo aprovechen solo agrega costo de escritura y espacio, y podría eliminarse del diseño final.
- Con tablas pequeñas el planificador puede estimar mal: en ambos modelos estimó 80 filas para `merchants` / `dim_merchant` cuando hay 8. No cambió el plan, pero conviene correr `ANALYZE` antes de medir para que las estimaciones sean confiables.

4. Conclusiones

1. El OLTP en 3FN es correcto para registrar pagos, pero obliga a joins y a cálculos por fila (`EXTRACT`, comparación de texto) en cada consulta analítica.
2. El esquema estrella mueve ese trabajo a la carga: atributos precalculados, claves enteras y dimensiones pequeñas. Con 50 000 filas la mejora es de 1.12× a 1.76× (mayor en Q3, donde desaparece el Sort previo a la agregación) y se amplía con el volumen.
3. SCD2 en `dim_customer` conserva la verdad histórica del segmento; sobrescribir sesgaría la tasa de fraude por segmento.
4. Los índices no son gratuitos ni universales: su utilidad depende de la selectividad del filtro, y se justifica con `EXPLAIN ANALYZE`, no por intuición.